41039 Programming 1

Revision of Week 4: methods, functions, the stack and the heap, records, and a catch-up on lists, dicts and tuples



Dr. Xuan-Bach Le
le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the next hour works

Same deal as before. Not a test. Nothing is recorded against you. Answering earns extra credit, and being wrong out loud earns it too.

Part 1 — on your own

- Nine short questions, about a minute each: six on Week 4, then three on lists, dicts and tuples
- Commit to an answer **before** the reveal: on paper, or out loud

Part 2 — in teams of 3 or 4

- Four rounds: **draw the memory**, **spot the errors**, **predict the output**, **write the code**
- One sheet of paper, one spokesperson. Every team that reports back earns credit
- The last one goes to the keyboard: one Java team, one Python team

These are hard on purpose. Which variable is this? Which frame is it in? Does the caller ever see the change? If you are drawing boxes and arrows, you are doing it right.

For every method call: what is copied, and what is shared?

Outline

1. On your own
2. On your own: lists, dicts and tuples
3. Team round 1: draw the memory
4. Team round 2: spot the errors
5. Team round 3: predict the output
6. Team round 4: write the code
7. Wrapping up

Which of these compile?

Alone ▪ 90 seconds

```java
static int sign(int n) {              // (a)
    if (n > 0) return 1;
    else if (n < 0) return -1;
}
static boolean even(int n) {          // (b)
    return n % 2;
}
static void hello() { return; }       // (c)
static int twice(int n) {             // (d)
    return n * 2;
    System.out.println("done");
}
```

Which of these compile?

Alone ▪ 90 seconds

```java
static int sign(int n) {                // (a)
    if (n > 0) return 1;
    else if (n < 0) return -1;
}
static boolean even(int n) {            // (b)
    return n % 2;
}
static void hello() { return; }         // (c)
static int twice(int n) {               // (d)
    return n * 2;
    System.out.println("done");
}
```

Answers

(a) ✗ `missing return statement.` When `n` is 0, no `return` is reached. Add `else return 0;`

(b) ✗ `n % 2` is an `int`, not a `boolean`: `return n % 2 == 0;`

(c) ✓ a `void` method may `return` early, with no value. `return 0;` would not compile: `unexpected return value`

(d) ✗ `unreachable statement.` `return` ends the method; nothing after it can run

The compiler checks every path. It never checks that the answer is right.

Which one runs?

Alone ▪ 60 seconds

```
static void show(int x)
    { System.out.println("int " + x); }
static void show(double x)
    { System.out.println("double " + x); }
static void show(int x, int y)
    { System.out.println("two " + (x + y)); }
static void show(int... xs)
    { System.out.println("varargs " + xs.
     length); }
```

What prints?

`show(5); show(5.0); show(2, 3); show(1, 2, 3); show();`

And: could you add `static int show(int x)`?

Which one runs?

Alone ▪ 60 seconds

```
static void show(int x)
    { System.out.println("int " + x); }
static void show(double x)
    { System.out.println("double " + x); }
static void show(int x, int y)
    { System.out.println("two " + (x + y)); }
static void show(int... xs)
    { System.out.println("varargs " + xs.
     length); }
```

What prints?

`show(5);` `show(5.0);` `show(2, 3);` `show(1, 2, 3);` `show();`

And: could you add `static int show(int x)`?

Answers

`int 5` — the exact match wins

`double 5.0`

`two 5` — two `int`s: the two-parameter version

`varargs 3` — only varargs takes three

`varargs 0` — varargs accepts **zero** values

No: `method show(int) is already defined`. Overloads must differ in their **parameter lists**; the return type does not count

Three variables called `n`

Alone ▪ 90 seconds

1. What prints? **2.** Just before g returns: how many frames, and what is each `n`?

```
static int f(int n) {
    n = n * 2;
    return g(n) + n;
}
static int g(int n) {
    n = n + 1;
    return n;
}
public static void main(String[] args) {
    int n = 3;
    int r = f(n);
    System.out.println(n + " " + r);
}
```

Three variables called `n`

Alone ▪ 90 seconds

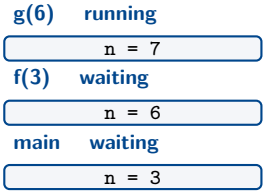
1. What prints? **2.** Just before `g` returns: how many frames, and what is each `n`?

```
static int f(int n) {
    n = n * 2;
    return g(n) + n;
}
static int g(int n) {
    n = n + 1;
    return n;
}
public static void main(String[] args) {
    int n = 3;
    int r = f(n);
    System.out.println(n + " " + r);
}
```

Answers

1. `3 13`. `f` doubles *its* `n` to 6; `g` makes *its* `n` 7 and returns 7; `f` returns $7 + 6$. `main`'s `n` was copied, never changed

2. Three frames, three separate `n`s:



Mutate, or reassign?

Alone ▪ 90 seconds

```
static void a(int[] v) { v[0] = 9; }
static void b(int[] v)
    { v = new int[] {7, 7}; }
static void c(int[] v) {
    v = new int[] {5, 5};
    v[1] = 1;
}
static int[] d(int[] v) {
    v = new int[] {4, 4};
    return v;
}
```

```
int[] x = {1, 2};
a(x);          // x is now?
b(x);          // x is now?
c(x);          // x is now?
x = d(x);      // x is now?
int[] y = x;  y[1] = 0;   // x?
```

Mutate, or reassign?

Alone ▪ 90 seconds

```
static void a(int[] v) { v[0] = 9; }
static void b(int[] v)
    { v = new int[] {7, 7}; }
static void c(int[] v) {
    v = new int[] {5, 5};
    v[1] = 1;
}
static int[] d(int[] v) {
    v = new int[] {4, 4};
    return v;
}
```

```
int[] x = {1, 2};
a(x);          // x is now?
b(x);          // x is now?
c(x);          // x is now?
x = d(x);      // x is now?
int[] y = x;  y[1] = 0;   // x?
```

Answers

a: `{9, 2}` — `v` is a copy of the *reference*
b: `{9, 2}` — reassigning `v` re-points only the copy
c: `{9, 2}` — `v[1] = 1` changed the *new* array
d: `{4, 4}` — `main` assigned the returned reference
`y[1] = 0`: `{4, 0}` — two names, one array

Python: what does the caller see?

Alone ▪ 90 seconds

```python
# (a)
def f(items, n):
    items += [n]
    n += 1
    items = items + [n]

nums = [1]
k = 5
r = f(nums, k)
print(nums, k, r)

# (b)
print(print("x"))
```

Python: what does the caller see?

Alone ▪ 90 seconds

```python
# (a)
def f(items, n):
    items += [n]
    n += 1
    items = items + [n]

nums = [1]
k = 5
r = f(nums, k)
print(nums, k, r)

# (b)
print(print("x"))
```

Answers

(a) `[1, 5] 5 None`

`items += [n]` **mutates** the list, so `nums` sees it.

`n += 1` makes a new number, for the local `n` only.

`items = items + [n]` builds a **new** list and rebinds the local name.

No `return`, so `r` is `None`

(b) `x`, then `None`. The inner `print` shows `x` and returns `None`; the outer one prints that

+= on a list mutates it. x = x + [...] builds a new one.

Records: compile, and print what?

Alone ▪ 60 seconds

```
record Point(int x, int y) {}

Point p = new Point(1, 2);
System.out.println(p);                  // (a)
System.out.println(p.x() + p.y());      // (b)
p.x = 5;                                // (c)
Point q = new Point(1);                 // (d)
p = new Point(3, 4);                    // (e)
System.out.println(p);
```

Records need Java 16 or later: run these on Ed, not in the site's console.

Records: compile, and print what?

Alone ▪ 60 seconds

```
record Point(int x, int y) {}

Point p = new Point(1, 2);
System.out.println(p);                  // (a)
System.out.println(p.x() + p.y());      // (b)
p.x = 5;                                // (c)
Point q = new Point(1);                 // (d)
p = new Point(3, 4);                    // (e)
System.out.println(p);
```

Records need Java 16 or later: run these on Ed, not in the site's console.

Answers

(a) `Point[x=1, y=2]` — printing comes free

(b) 3 — components are read with *methods*: `p.x()`

(c) ✗ `cannot assign a value to final variable x`. A record's components never change

(d) ✗ the constructor takes **both** components, in order

(e) ✓ prints `Point[x=3, y=4]`. The *Point* cannot change, but the *variable* `p` can refer to a different one

A final component is not a final variable. Know which one you have.

Outline

1. On your own
2. On your own: lists, dicts and tuples
3. Team round 1: draw the memory
4. Team round 2: spot the errors
5. Team round 3: predict the output
6. Team round 4: write the code
7. Wrapping up

Lists: one list, or two?

Alone ▪ 90 seconds

```
a = [1, 2]
b = a
a.append([3, 4])
b += [5]
c = a + [6]
c[0] = 0
print(a)
print(len(a), len(c))
a[1:3] = ['x']
print(b)
```

What prints?

Three lines. And how many lists are there at the end?

Lists: one list, or two?

Alone ▪ 90 seconds

```
a = [1, 2]
b = a
a.append([3, 4])
b += [5]
c = a + [6]
c[0] = 0
print(a)
print(len(a), len(c))
a[1:3] = ['x']
print(b)
```

What prints?

Three lines. And how many lists are there at the end?

Answers

`[1, 2, [3, 4], 5]` `4 5` `[1, 'x', 5]`

`b = a`: two names, **one** list

`append` adds **one** element, here a whole list

`b += [5]` mutates that same list

`a + [6]` is a **new** list: `c[0] = 0` leaves `a` alone

`a[1:3] = ['x']`: two elements out, one in

Two lists at the end: `a` and `b` share one, `c` is the other

append adds one element. + builds a new list. += changes the one you have.

Dicts: through a function

Alone ▪ 90 seconds

```python
def add_mark(book, name, mark):
    book[name] = mark

def rename(book):
    book = {'nobody': 0}

marks = {'Linh': 7, 'Bach': 9}
add_mark(marks, 'Bach', 10)
add_mark(marks, 'An', 8)
rename(marks)
print(len(marks), marks)
print('An' in marks, 8 in marks)
print(marks['Chi'])
```

Dicts: through a function

Alone ▪ 90 seconds

```
def add_mark(book, name, mark):
    book[name] = mark

def rename(book):
    book = {'nobody': 0}

marks = {'Linh': 7, 'Bach': 9}
add_mark(marks, 'Bach', 10)
add_mark(marks, 'An', 8)
rename(marks)
print(len(marks), marks)
print('An' in marks, 8 in marks)
print(marks['Chi'])
```

Answers

`3 {'Linh': 7, 'Bach': 10, 'An': 8}`
`True False`, then `KeyError: 'Chi'`

`book[name] = mark` **mutates** the caller's dict: `'Bach'` changes, `'An'` is added. Keys are unique, so three entries

`rename` only **rebinds** its own `book`, like Java's `b(x)`

`in` asks about **keys**; 8 is a value

A missing key is an error, not `None`: test `'Chi' in marks` first

d[k] = v changes or adds. Reading a missing key is a KeyError.

Tuples: works, or TypeError?

Alone ▪ 90 seconds

```
point = (3, 4)
pairs = [(1, 'a'), (2, 'b')]

x, y = point              # (a)
point[0] = 10             # (b)
point = (10, y)           # (c)
pairs[0] = (9, 'z')       # (d)
pairs[1][0] = 9           # (e)
one = (5)                 # (f) type(one)?
x, y = y, x               # (g) x and y now?
```

Judge each line on its own, as if it came straight after (a).

Tuples: works, or TypeError?

Alone ▪ 90 seconds

```
point = (3, 4)
pairs = [(1, 'a'), (2, 'b')]

x, y = point              # (a)
point[0] = 10             # (b)
point = (10, y)           # (c)
pairs[0] = (9, 'z')       # (d)
pairs[1][0] = 9           # (e)
one = (5)                 # (f) type(one)?
x, y = y, x               # (g) x and y now?
```

Judge each line on its own, as if it came straight after (a).

Answers

(a) ✓ `x` is 3, `y` is 4

(b) ✗ `TypeError`: a tuple does not support item assignment

(c) ✓ a **new** tuple; the variable is rebound, like `p = new Point(3, 4)`

(d) ✓ the **list** changes: a new tuple in slot 0

(e) ✗ that is *inside* a tuple: `TypeError`

(f) `int`. The comma makes a tuple: `(5,)`

(g) `4 3`. The right side becomes `(4, 3)` first, then is unpacked

A tuple cannot change. The variable or list holding it can.

Outline

1. On your own
2. On your own: lists, dicts and tuples
3. Team round 1: draw the memory
4. Team round 2: spot the errors
5. Team round 3: predict the output
6. Team round 4: write the code
7. Wrapping up

Form teams

Now

- Teams of **3 or 4**, with the people next to you
- One sheet of paper. Write the team's names on it
- Pick a **spokesperson** — a different one for each challenge, if you can

For each challenge

- The timer is on the slide
- When it runs out, one team is called on to answer. Others may disagree — **disagreeing with a reason also earns credit**
- "I think it's X *because*..." beats "X"

How to draw memory

Stack on the left: one box per method call, newest on top, holding that call's parameters and local variables. Heap on the right: every array or object made with `new`. An arrow for every reference. Numbers and `booleans` sit in the box itself.

Draw it: the stack and the heap

Teams ▪ 5 min

```
static int[] tally(int[] marks, int pass) {
    int[] counts = new int[2];
    for (int m : marks) {
        if (m >= pass) counts[1]++;
        else counts[0]++;
    }
    // (*) draw the stack and the heap HERE
    return counts;
}
public static void main(String[] args) {
    int[] marks = {72, 45, 91};
    int pass = 50;
    int[] result = tally(marks, pass);
    System.out.println(result[0] + " fail");
    System.out.println(result[1] + " pass");
}
```

Draw it, then answer

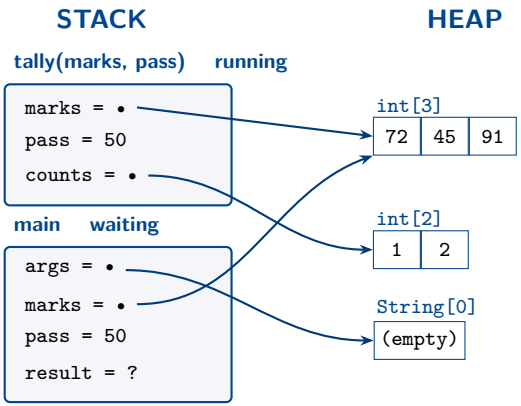
1. Draw the stack and the heap at `(*)`. Every variable, every array, every arrow.

2. How many variables are called `marks`? How many arrays hold 72, 45, 91?

3. Where is `m` at `(*)`?

4. Redraw it just after `tally` returns. What has gone, and what survived? What prints?

Draw it: one answer



2. Two variables called `marks`, one in each frame — and **one** array. The call copied the reference, not the array. Two passes, each holding its own 50

3. Nowhere. `m` belonged to the `for` loop, and the loop has ended

4. `tally`'s frame is popped: its `marks`, `pass` and `counts` are gone. The `int[2]` **survives** — `result` now refers to it. Prints `1 fail, 2 pass`

Frames die on return. Arrays live as long as something refers to them.

Outline

1. On your own
2. On your own: lists, dicts and tuples
3. Team round 1: draw the memory
4. Team round 2: spot the errors
5. Team round 3: predict the output
6. Team round 4: write the code
7. Wrapping up

Spot the errors 1: marks, in Java

Teams ▪ 5 min

✗ Should print the average and the highest mark, then zero every mark.

```
1  public class Marks {
2      static average(int[] values) {
3          int total = 0;
4          for (int v : values) total += v;
5          return total / values.length;
6      }
7      static void highest(int[] values) {
8          int max = 0;
9          for (int v : values) if (v > max)
       max = v;
10         return max;
11     }
12     static void clear(int[] values) {
13         values = new int[values.length];
14     }
15     public static void main(String[] args) {
16         int[] marks = {72, 55, 91, 64};
17         System.out.println(average(marks)
18             + " " + highest(marks));
19         clear(marks);
20         System.out.println(marks[0]);
21         System.out.println(total);
22     }
23 }
```

Three tasks

1. Five bugs: line numbers? **2.** `javac` reports three, not all at once: how many rounds?
3. Compile errors fixed: what prints?

Spot the errors 1: answers

The five bugs

Round 1 — `javac` stops at the first:
L2 no return type: `return type required`. It should be `double`

Round 2 — three messages, two bugs:
L7, L10 `void` but returns `max`: `unexpected return value`, and on L18 `'void' type not allowed here`
L21 `total` lived in `average`'s frame: `cannot find symbol`

Never reported:
L5 `int / int` gives 70: `(double) total / values.length`
L13 reassigning `values` leaves the caller's array alone: set each `values[i] = 0`

3. Compiles, still wrong

```
70.0 91
72
```

`70.0`: divided in `int`s, *then* widened.
`72`: `clear` changed nothing the caller can see.

A sixth, for the keen: `max = 0` works only because marks are never negative. Start from `values[0]`.

Spot the errors 2: marks, in Python

Teams ▪ 4 min

✗ curved must return a NEW list, and leave marks alone.

```
1  def average(values):
2      total = 0
3      for v in values:
4          total += v
5      print(total / len(values))
6  def curved(marks, bonus=5):
7      result = marks
8      for i in range(len(result)):
9          result[i] += bonus
10     return result
11 def report(*names, title):
12     for n in names:
13         print(title + ": " + n)
```

```
14 marks = [72, 55, 91]
15 avg = average(marks)
16 print("Average is " + avg)
17 new_marks = curved(bonus=10, marks)
18 print(marks, new_marks)
19 report("Linh", "Bach", "Results")
```

Should print: `Average is 72.666…`, `[72, 55, 91] [82, 65, 101]`, `Results: Linh`, `Results: Bach`

Two tasks

1. Five errors (line numbers).
2. In what **order** does Python show them? Which never shows one?

Spot the errors 2: answers, in the order you meet them

Run, fix, run again

1. **L17** `SyntaxError`: positional argument follows keyword argument. Nothing runs at all
2. **L5** `print`, not `return`: `72.66…` appears, then L16 fails — `str + NoneType`
3. **L16** still a `TypeError`, now `str + float`. Use a comma
4. **L19** missing keyword-only argument `'title': *names` swallowed `"Results"`
5. **L7** ✗ **no message.** `result = marks` is a second name for the **same** list: L18 prints `[82, 65, 101]` twice

✓ The fixes, L5, L7, L16, L17, L19

```
    return total / len(values)
    result = marks[:]
print("Average is", avg)
new_marks = curved(marks, bonus=10)
report("Linh", "Bach",
       title="Results")
```

`marks[:]` slices the whole list: a **new** list, same elements.

The silent bug is the one about references. It usually is.

Outline

1. On your own
2. On your own: lists, dicts and tuples
3. Team round 1: draw the memory
4. Team round 2: spot the errors
5. Team round 3: predict the output
6. Team round 4: write the code
7. Wrapping up

Predict 1: grow

Teams ▪ 4 min

```
static int[] grow(int[] a, int extra) {
    int[] b = new int[a.length + 1];
    for (int i = 0; i < a.length; i++)
        b[i] = a[i];
    b[a.length] = extra;
    a[0] = -1;
    extra = 0;
    return b;
}
static void show(int[] v) {
    for (int n : v)
        System.out.print(n + " ");
    System.out.println();
}
```

```
int[] x = {1, 2};
int e = 3;
int[] y = grow(x, e);
int[] z = y;
z[1] = 99;
show(x);
show(y);
System.out.println(e);
System.out.println(x == y);
System.out.println(y == z);
```

Two tasks

1. Write the five lines of output.

2. How many arrays exist at the end? Which variables refer to each?

Predict 1: answers

1. Output

```
-1 2
1 99 3
3
false
true
```

2. Two arrays

`x` → `{-1, 2}`

`y`, `z` → `{1, 99, 3}`

Why

`-1 2`: `a[0] = -1` followed the copied reference to `x`'s array. It ran *after* the copy, so `y` still starts with 1

`1 99 3`: `z = y` copied a reference, so `z[1] = 99` is `y[1] = 99`

`3`: `extra = 0` changed `grow`'s copy. `e` is an `int`, copied by value

`false`: `==` on arrays compares **references** — `grow` made a new array

`true`: `y` and `z` refer to the same one

`==` asks "the same array?", not "the same numbers?".

Predict 2: pad and shout

Teams ▪ 4 min

```
def pad(items, n, fill=0):
    while len(items) < n:
        items.append(fill)
    return len(items)

def shout(word):
    word = word + "!"
    return word
```

```
a = [1]
b = a
c = pad(b, 3)
d = pad(a[:], 5, fill=9)
w = "hi"
shout(w)
print(a, b, c, d, w)
print(pad(a, 2))
print(shout(w) + shout(w))
```

Two tasks

1. Write the three lines of output.
2. Which calls changed `a`? Which changed `w`?

Predict 2: answers

1. Output

```
[1, 0, 0] [1, 0, 0] 3 5 hi
3
hi!hi!
```

2. Only `pad(b, 3)` changed `a`. Nothing ever changed `w`.

Why

`pad(b, 3)`: `b` is `a` — one list. `append` mutates it, so `a` and `b` both show `[1, 0, 0]`

`pad(a[:], 5, fill=9)`: the slice is a **copy**. Five nines went into a list nobody kept; `d` is 5, `a` untouched

`shout(w)`: strings are immutable. `word = word + "!"` rebinds the local name, and the result was thrown away

`pad(a, 2)`: already length 3 — zero passes, returns 3

A returned value you do not keep is simply gone.

Predict 3: count the words

Teams ▪ 4 min

```
def tally(words):
    counts = {}
    for w in words:
        if w in counts:
            counts[w] += 1
        else:
            counts[w] = 1
    return counts

def best(counts):
    top = ('', 0)
    for pair in counts.items():
        if pair[1] > top[1]:
            top = pair
    return top
```

```
words = "to be or not to be".split()
c = tally(words)
w, n = best(c)
print(len(words), len(c))
print(c)
print(w, n)
c['be'] += 1
print(best(c))
words.append('to')
print(c['to'])
```

Two tasks

1. Write the five lines of output.

2. `to` and `be` both appear twice. Which one does `best` pick first, and why? What one-character change picks the other?

Predict 3: answers

1. Output

```
6 4
{'to': 2, 'be': 2, 'or': 1, 'not': 1}
to 2
('be', 3)
2
```

2. The tie `>` keeps the **first** pair to reach 2, and `'to'` went in first. With `>=` a later tie replaces it, and line 3 becomes `be 2`.

Why

`6 4`: six words, four keys — keys are unique

`tally` builds a **new** dict. Its frame dies; the dict survives in `c`

`items()` gives `(key, value)` tuples. `top = pair` rebinds `top`: no tuple changes

`w, n = best(c)`: one return, two values

`c['be'] += 1` reads 2, stores 3: `be` wins outright

`words.append('to')` changes the list only. Nothing links it to `c`

Counting is a dict's job. Handing back a pair is a tuple's.

Outline

1. On your own
2. On your own: lists, dicts and tuples
3. Team round 1: draw the memory
4. Team round 2: spot the errors
5. Team round 3: predict the output
6. Team round 4: write the code
7. Wrapping up

Code it 1: reverse, two ways

Teams ▪ 4 min

On paper, in Java:

- `static int[] reversed(int[] v)` — returns a **new** array, and leaves `v` alone. A **function**
- `static void reverseInPlace(int[] v)` — reverses `v` itself, and returns nothing. A **procedure**

For `{1, 2, 3, 4, 5}`: `reversed` gives `{5, 4, 3, 2, 1}` and the original is unchanged; after `reverseInPlace`, the original *is* `{5, 4, 3, 2, 1}`.

Then explain these two

A. A classmate writes `reverseInPlace(int[] v) { v = reversed(v); }`. The array does not change. Why?

B. Another swaps `v[i]` with `v[v.length - 1 - i]`, for `i` from 0 to `v.length - 1`. For `{1, 2, 3, 4}` the array comes out `{1, 2, 3, 4}`. Why?

Code it 1: answers

✓ Function: a new array

```
static int[] reversed(int[] v) {
    int[] r = new int[v.length];
    for (int i = 0; i < v.length; i++)
        r[i] = v[v.length - 1 - i];
    return r;
}
```

✓ Procedure: swap elements, half way

```
static void reverseInPlace(int[] v) {
    for (int i = 0; i < v.length / 2; i++) {
        int t = v[i];
        v[i] = v[v.length - 1 - i];
        v[v.length - 1 - i] = t;
    }
}
```

Why A and B fail

A. `v = reversed(v)` re-points `reverseInPlace`'s *copy* of the reference at the new array. The caller's variable still refers to the old one, untouched — the swap that could not work

B. Going all the way swaps every pair **twice**: the second half of the loop undoes the first. Stop at `v.length / 2` — which, for an odd length, leaves the middle element where it belongs

Code it 2: summarise

Teams ▪ 6 min

Write one method that returns **three** answers: the smallest, the largest, and the mean. One pass over the data, no sorting.

- **Java team** (`Stats.java`): a `record Summary(int min, int max, double mean)`, and `static Summary summarise(int[] values)`
- **Python team** (`stats.py`): `summarise(values)` returns a tuple. No `min()`, `max()` or `sum()` — write the loop
- Everyone else writes both on paper, and shouts fixes

Run it with all of these

input	min	max	mean
`72, 55, 91, 64`	55	91	70.5
`5`	5	5	5.0
`-3, -7`	-7	-3	-5.0

Java prints `Summary[min=55, max=91, mean=70.5]`; Python prints `(55, 91, 70.5)`.

Code it 2: one answer in each language

✓ Java

```java
record Summary(int min, int max,
               double mean) {}

static Summary summarise(int[] values) {
    int min = values[0];
    int max = values[0];
    int total = 0;
    for (int v : values) {
        if (v < min) min = v;
        if (v > max) max = v;
        total += v;
    }
    return new Summary(min, max,
        (double) total / values.length);
}
```

✓ Python

```python
def summarise(values):
    lo = values[0]
    hi = values[0]
    total = 0
    for v in values:
        if v < lo:
            lo = v
        if v > hi:
            hi = v
        total += v
    mean = total / len(values)
    return (lo, hi, mean)
```

One method, one return, three answers: bundle them.

Code it 2: three ways to get it wrong

✗ Start at 0

`min = 0` gives `min=0` for the first test. `max = 0` gives `max=0` for `{-3, -7}`. Start from `values[0]`: a value that is really in the data.

✗ `total / values.length`

In Java, `int / int` gives `mean=70.0` for the first test. Storing it in a `double` is too late: the fraction was already thrown away. Cast *first*.

✗ `print`, not `return`

In Python, printing the tuple inside the function *looks* right — then `print(summarise(…))` adds a `None`. The caller gets nothing it can use.

The -3, -7 test is the one that catches the first mistake. That is why it is there.

Outline

1. On your own
2. On your own: lists, dicts and tuples
3. Team round 1: draw the memory
4. Team round 2: spot the errors
5. Team round 3: predict the output
6. Team round 4: write the code
7. Wrapping up

You should now be able to...

Week 4

- ✓ say why a method does not compile: return type, a **reachable** `return`
- ✓ predict which overload runs
- ✓ draw the **stack and the heap** at any line
- ✓ explain why Java is **pass-by-value**, and what the value is for an array
- ✓ tell **mutating** an argument from **rebinding** a parameter, in Java and in Python
- ✓ use defaults, keyword arguments and `*names`
- ✓ tell a function that `returns` from one that only prints
- ✓ return several values as one, with a **record** or a tuple

Lists, dicts and tuples

- ✓ say what `append`, `+`, `+=` and a slice do to a list
- ✓ change, add and test **keys** in a dict, and expect a `KeyError`
- ✓ unpack a tuple, and tell the tuple from the variable holding it

If any box is unticked

Week 4 page, sections 01–08, the quiz and the exercises:
`lexuanbach.github.io/41039/weeks/week-4.html`
Lists, dicts and tuples: Weeks 2+3 page, sections 09–10:
`lexuanbach.github.io/41039/weeks/week-2-3.html#s09`